Software Requirements Specification (SRS)

Project: College Timetable Generation System
Version: 1.0
Authors: SAATHVIK SAI SUVARNA-PES1UG24AM238

AYUSH PM -PES1UG24AM909

RISHI BHARADWAJ – PES1UG24AM223

ROHIT CHAKI – PES1UG24AM228

Date: 07-10-2026
Status: Approved / Baseline

Revision history

Version	Date	Author	Change summary	Approval
1.0	07-10-2026	SAATHVIK SAI SUVARNA-PES1UG24AM238 AYUSH PM -PES1UG24AM909 RISHI BHARADWAJ – PES1UG24AM223 ROHIT CHAKI – PES1UG24AM228	Initial comprehensive SRS baseline for College Timetable Generation System	Course Coordinator

Approvals

Role	Name	Signature / Email	Date
Course Coordinator	Dr. Faculty Evaluator	evaluator@university.edu	07-10-2026

Lead Developer	Project Team	team@university.edu	07-10-2026

Table of Contents

1. Introduction
2. Overall description
3. External interfaces
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. UML Use-Case Diagram
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document specifies the software requirements for the College Timetable Generation System. It establishes both functional and non-functional specifications, mathematical constraints, external interfaces, and verification targets for developers, academic evaluators, and system testers.

1.2 Scope

The system automates the allocation of academic courses, designated faculty members, lecture classrooms, practical laboratories, and student sections into conflict-free weekly time slots. The scope encompasses administrative data entry, constraint configuration, the automated optimization solver (Google OR-Tools CP-SAT), interactive manual adjustment grids, role-based timetable viewing, and export utilities. Hardware device management and central university ERP fee/admission databases are outside the direct boundary of this system.

1.3 Audience

- Academic Coordinators and Department Heads (primary users)
- Faculty Members and Students (viewers)
- Software Development and Quality Assurance Teams
- Academic Project Evaluators

1.4 Definitions and Acronyms

- CSP: Constraint Satisfaction Problem
- CP-SAT: Constraint Programming - Boolean Satisfiability (Google OR-Tools solver engine)
- FR / NFR: Functional Requirement / Non-Functional Requirement
- Hard Constraint: A physical or logical condition that must never be violated under any circumstance
- Soft Constraint: A preference or quality metric minimized or maximized to produce an optimal schedule
- Slot: A standard discrete time interval within an academic day (e.g., 50 minutes or 60 minutes)
- RTM: Requirements Traceability Matrix
- RBAC: Role-Based Access Control

2. Overall description

2.1 Product perspective

The College Timetable Generation System is a web-based scheduling platform. It replaces error-prone, manual spreadsheet-based scheduling with a mathematical constraint programming engine accessible via modern web browsers. The system operates on a client-server architecture with an independent asynchronous solver worker.

2.2 Major product functions (detailed)

- Resource registry management (courses, rooms, labs, faculty, student sections).
- Rule and constraint definitions (institutional timings, lunch intervals, workload caps).
- Automated timetable generation using constraint satisfaction logic.
- Conflict detection and mathematical infeasibility diagnostic reporting.
- Interactive drag-and-drop timetable adjustment interface with real-time clash verification.
- Multi-dimensional schedule views (by Section, Faculty, and Classroom/Lab).
- Exporting schedules into formatted PDF and spreadsheet formats.

2.3 User roles and characteristics

- Academic Administrator / Coordinator: Configures institutional parameters, defines faculty workloads, runs generation jobs, resolves conflicts, and publishes final schedules. Requires full system

privileges.
- Faculty Member: Submits scheduling preferences, views assigned teaching allocations, and downloads individual schedules. Requires read access with personal schedule filtering.
- Student: Views section-specific weekly schedules, classroom locations, and timing updates without administrative privileges.

2.4 Operating environment
- Server Environment: Ubuntu Linux 22.04 LTS / Docker container, Python 3.10+ runtime, FastAPI backend.
- Database: Relational storage using PostgreSQL or SQLite.
- Client Environment: Modern evergreen web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) across standard desktop and laptop resolutions.

2.5 Constraints
- Mathematical complexity: The academic timetabling problem is NP-hard; solutions must converge or declare infeasibility within bounded time.
- Institutional policy limits: Faculty maximum daily lecture ceilings, fixed institutional lunch hours, and room capacity thresholds must be strictly respected.
- Open-source software constraints: Use open-source libraries without proprietary enterprise licensing restrictions.

2.6 Assumptions and Dependencies
- Standardized Time Discretization: Assumes all academic schedules follow uniform discrete time blocks (e.g., 50 or 60-minute slots) across institutional working days (Monday–Friday/Saturday).
- Data Finalization Prior to Run: Assumes faculty course allocations and student batch strengths are finalized before running automated master timetable generation.
- Single Campus Deployment: Assumes travel time between lecture halls does not require inter-campus transit constraints.

3. External interface requirements

3.1 User interfaces
- Admin Dashboard: Administrative screens for managing entities (Faculty, Rooms, Courses, Batches).
- Timetable Master Grid: Responsive matrix displaying Days (Monday-Friday/Saturday) on the vertical axis and Time Slots on the horizontal axis with color-coded subject cards.
- Filtering Bar: Dynamic dropdown selectors to switch views across Sections, Faculty members, and Rooms.

- Clash Warning Alerts: Visual modal dialogs detailing exact rule conflicts upon manual drag-and-drop modifications.

3.2 Hardware interfaces

The software operates on standard cloud or on-premise compute servers (minimum 2 vCPU, 4 GB RAM) and renders on standard computing devices (monitors, laptops, tablets) via standard web viewports.

3.3 Software interfaces

- Constraint Optimization Engine: Google OR-Tools (CP-SAT) via Python library.
- Data Storage Interface: SQLAlchemy ORM interfacing with PostgreSQL/SQLite for relational entities.
- Export Engines: Headless PDF rendering (e.g., WeasyPrint or ReportLab) and OpenPyXL for Excel spreadsheet generation.

3.4 Communications

- Secure Hypertext Transfer Protocol (HTTPS with TLS 1.3) enforced for all client-server communications.
- RESTful JSON APIs for data transfer between client frontend and Python server.

4. System features (detailed)

Each requirement below includes acceptance criteria and a reference test case. IDs follow TT-F-###.

4.1 Resource & Curriculum Management

Description: Manage the academic master data (courses, faculty, rooms and laboratories, and student sections) that forms the input to the scheduling engine.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
TT-F-001	The system shall allow administrators to create, update, and delete	Functional	High	Coordinator	AC-TT-F-001: Course entries persist in database	Prerequisite for all scheduling logic.

	academic courses with credit values, weekly lecture hours, and course types (Theory vs. Practical).				with verified credit parameters. Test: TC-RES-01	
TT-F-002	The system shall record faculty profiles including maximum teaching hours per week, assigned subjects, and unavailability /leave time slots.	Functional	High	Faculty / Coordinator	AC-TT-F-002: Faculty availability matrix correctly flags blocked slots. Test: TC-RES-02	Must link to Course entities.
TT-F-003	The system shall manage classrooms and specialized laboratories, including seating capacities, room numbers, and supported equipment flags.	Functional	High	Operations / Admin	AC-TT-F-003: Room attributes and seating capacity limits are validated upon entry. Test: TC-RES-03	Used for capacity validation.
TT-F-004	The system shall allow administrators to define student	Functional	High	Coordinator	AC-TT-F-004: Sections can be assigned	Baseline entity for schedule grids.

	sections, student enrollment counts, and academic semester associations.				lists of required courses. Test: TC-RES-04	

4.2 Constraint Configuration

Description: Enforce the hard constraints that every generated or manually edited timetable must satisfy.

Req ID	Requirement (shall...)	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
TT-F-005	The system shall enforce hard constraints preventing any faculty member from being assigned to more than one classroom or section in any single time slot.	Functional	High	Coordinator / Faculty	AC-TT-F-005: Zero instances of double-booked faculty appear in generated schedules. Test: TC-CST-01	Hard Constraint.
TT-F-006	The system shall enforce hard constraints	Functional	High	Operations	AC-TT-F-006: Zero room collisions exist in any	Hard Constraint.

	preventing any room or laboratory from hosting multiple sessions concurrently.				generated master schedule. Test: TC-CST-02	
TT-F-007	The system shall ensure that student sections are never scheduled for more than one class concurrently, and assigned room capacity meets or exceeds section strength.	Functional	High	Students / Coordinator	AC-TT-F-007: Room capacity ≥ student count, section clash count equals zero. Test: TC-CST-03	Hard Constraint.
TT-F-008	The system shall schedule practical laboratory sessions in consecutive contiguous slots	Functional	High	Department	AC-TT-F-008: Lab courses are placed in adjacent unbroken periods without slot splitting.	Requires slot index grouping.

	(e.g., 2 or 3 consecutive hours) exclusively in designated lab venues.				Test: TC-CST-04	

4.3 Automated Generation Engine

Description: Generate a complete weekly timetable automatically using Google OR-Tools CP-SAT, and report infeasibility when no solution exists.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
TT-F-009	The system shall execute an automated solving routine via Google OR-Tools CP-SAT to output a complete weekly timetable satisfying all configured hard constraints.	Functional	High	Coordinator	AC-TT-F-009: Solver produces a 100% clash-free assignment matrix across all active sections. Test: TC-GEN-01	Core algorithm requirement.
TT-F-010	If a conflict-free timetable is mathematically impossible under the current constraints, the system	Functional	High	Coordinator	AC-TT-F-010: System returns INFEASIBLE status code with explanat	Prevents unbounded solver lockups.

	shall terminate gracefully and return an infeasibility diagnostic explaining conflicting parameters.				ory bottlenecks instead of timing out. Test: TC-GEN-02	
TT-F-011	The generation engine shall optimize soft constraints to minimize faculty idle gaps between lectures and prevent consecutive heavy theory courses for students.	Functional	Medium	Faculty / Students	AC-TT-F-011: Objective function penalty decreases with reduced idle waiting periods. Test: TC-GEN-03	Optimization objective.

4.4 Timetable Verification & Manual Modification

Description: View, verify and manually adjust timetables with real-time clash validation, and pin fixed slots before generation.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
TT-F-012	The system shall render interactive timetable views filterable by Student Section, Faculty Member, and Classroom.	Functional	High	All Users	AC-TT-F-012: Selected filter instantly displays corresponding weekly schedule grid.	Read/Interactive UI.

					Test: TC-UI-01	
TT-F-013	The system shall provide an interactive drag-and-drop editor allowing coordinators to manually swap or reassign class slots with immediate conflict validation.	Functional	High	Coordinator	AC-TT-F-013: Manual move triggers real-time clash check; invalid moves display immediate warning dialog. Test: TC-MOD-01	Validation engine check.
TT-F-014	The system shall permit administrators to lock ("pin") specific fixed course slots prior to running automated generation.	Functional	Medium	Coordinator	AC-TT-F-014: Pinned slots remain unchanged across subsequent solver executions. Test: TC-MOD-02	Modifies solver domain bounds.

4.5 Publishing & Exporting

Description: Export finalized schedules to printable and spreadsheet formats.

Req ID	Requirement	Type	Priority	Source/Stakeholder	Acceptance criteria / Test case ref	Comments / Dependencies
TT-F-015	The system shall export finalized	Functional	High	All Users	AC-TT-F-015: Generat	Reporting module.

	schedules to formatted printable PDF documents and downloadable Microsoft Excel (.xlsx) workbooks.				ed files mirror UI grid formatting and print cleanly on A4 landscape. Test: TC-EXP-01	

5. Non-functional requirements (detailed)

NFRs below are measurable and tied to test plans. IDs TT-NF-###.

Req ID	Requirement	Category	Priority	Acceptance criteria / Measurement
TT-NF-001	The automated solver shall generate a conflict-free timetable for up to 30 academic sections in ≤ 15 seconds under standard server loads.	Performance	High	Solver benchmark execution time ≤ 15 seconds. Test: TC-PERF-01
TT-NF-002	The system shall maintain ≥ 99.5% operational	Reliability	High	Production uptime tracking logs confirm ≥ 99.5%

	uptime during active academic scheduling and semester onboarding cycles.			availability per month. Test: TC-REL-01
TT-NF-003	The UI grid shall comply with WCAG 2.1 AA accessibility standards, supporting keyboard navigation and high-contrast color indicators for distinct subjects.	Usability / Accessibility	Medium	Automated Lighthouse accessibility score ≥ 90; pass color contrast audits. Test: TC-UX-01
TT-NF-004	The architecture shall isolate the optimization solver from the HTTP request thread using	Scalability / Maintainability	High	Web UI remains interactive and polls job status during heavy generation runs. Test: TC-PERF-02

	asynchronous background processing to prevent HTTP timeouts.			
TT-NF-005	The system shall record audit logs capturing all administrative modifications, manual slot swaps, and generation attempts with user IDs and timestamps.	Auditability	High	Immutable audit trail recorded in database; queryable by administrator. Test: TC-AUD-01

5.1. Security

5.1.1 Security Objectives

- Confidentiality and Access Control: Prevent unauthorized access to institutional staff schedules, personal leave allocations, and unreleased academic timetables.
- Schedule Integrity: Protect published timetables and room assignments against unauthorized modification, tampering, or privilege escalation.

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance criteria / Test case ref
TT-SR-001	All communications between client browsers	Security	High	HTTP requests automatically

	and backend APIs shall be encrypted using TLS 1.3.			redirect to HTTPS; SSL Labs rating grade A. Test: TC-SEC-01
TT-SR-002	The system shall implement Role-Based Access Control (RBAC), restricting schedule generation, manual overrides, and resource editing exclusively to authenticated Coordinators.	Security	High	Unauthorized API calls return 403 Forbidden. Test: TC-SEC-02
TT-SR-003	User passwords shall be stored using industry-standard salted cryptographic hashes (Argon2id or BCrypt with a minimum cost factor of 12).	Security	High	Zero plaintext passwords stored in database or traces in logs. Test: TC-SEC-03
TT-SR-004	All user inputs (names, room codes, course IDs) shall undergo strict server-side validation and parameterized queries to prevent SQL Injection and XSS attacks.	Security	High	OWASP ZAP automated security scan reveals 0 High/Critical injection vulnerabilities. Test: TC-SEC-04
TT-SR-005	The system shall enforce rate limiting on schedule generation triggers (maximum 5 generation requests per minute per IP) to mitigate Denial of Service (DoS) risks.	Security	Mediu m	Excessive generation requests return 429 Too Many Requests. Test: TC-SEC-05

6. Quality attributes & Acceptance tests

Acceptance Criteria:

- All 15 functional requirements (TT-F-001 to TT-F-015) successfully verified with passing test cases.
- 100% elimination of hard constraint conflicts in all generated schedules.
- All high-priority NFRs and all 5 Security Requirements passed without critical vulnerability disclosures.

Acceptance Test Suites:

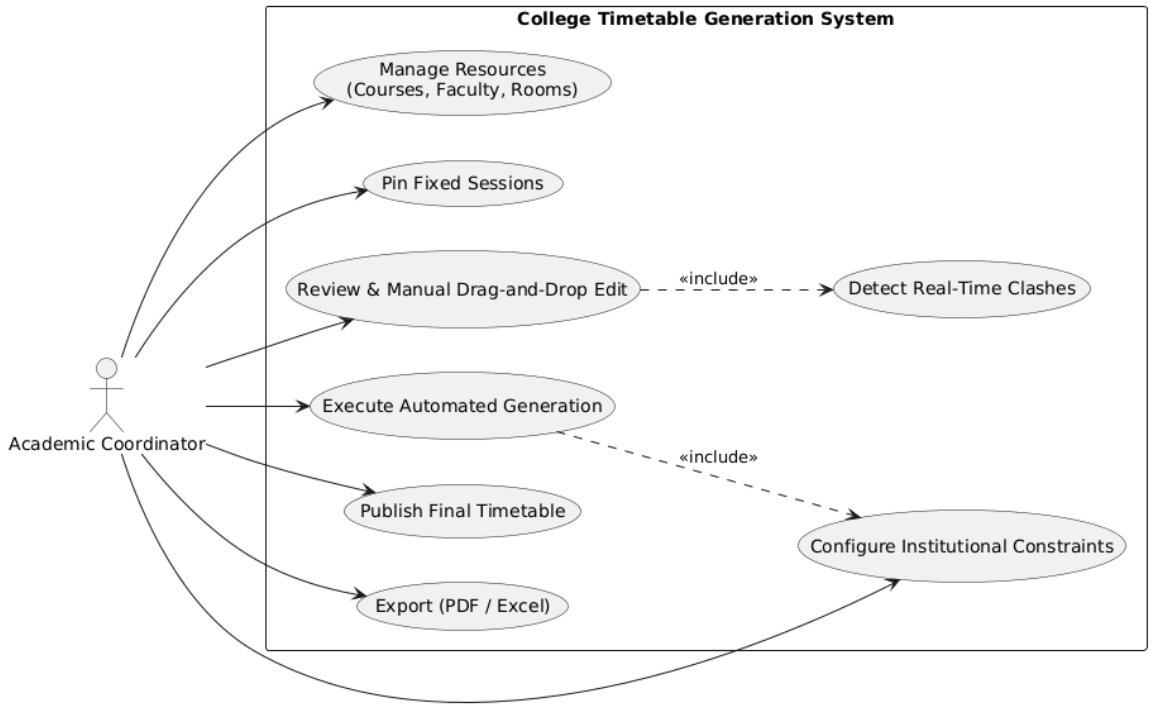
- Resource Validation Suite: Verifies CRUD behavior and data integrity

constraints.
- Solver Verification Suite: Feeds known overlapping scenarios to verify hard-constraint compliance and infeasibility detection.
- Manual Modification & Conflict Suite: Tests drag-and-drop reassignment and instantaneous collision detection.
- Security & Role Boundary Suite: Validates authorization barriers between Admin, Faculty, and Student access profiles.

7. System models and diagrams

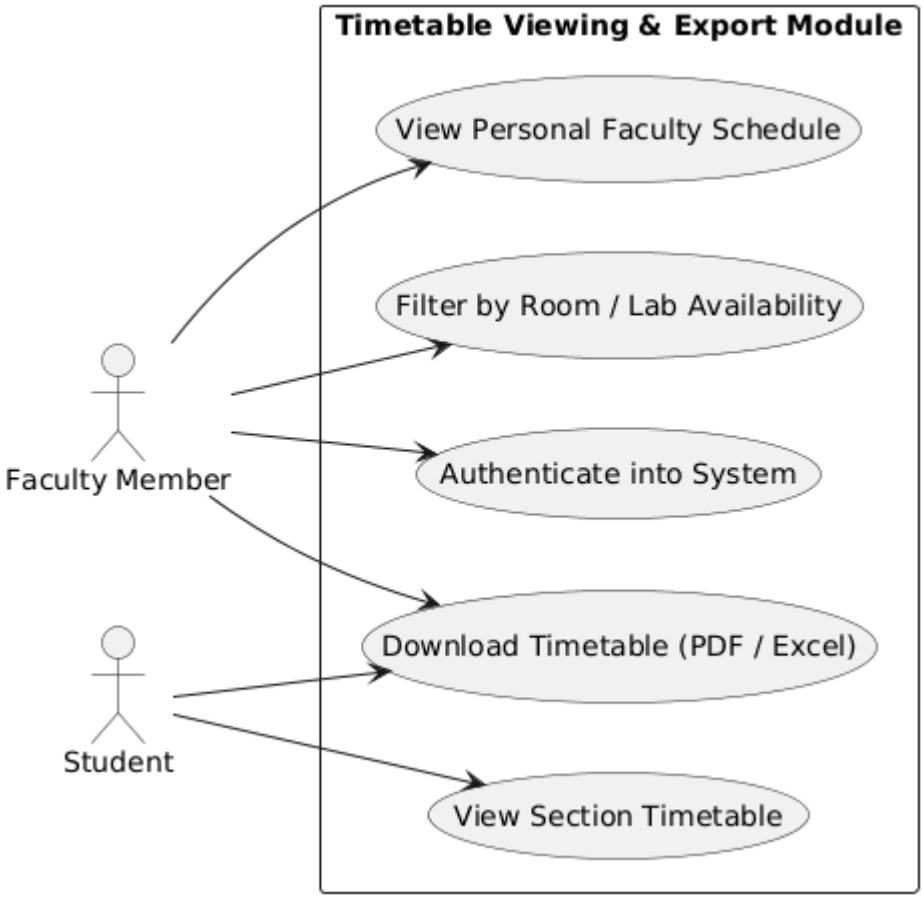
7.1 Use-Case Diagram 1: Administrative Configuration and Schedule Generation

This diagram captures coordinator interactions with the system setup, solver execution, and publishing modules.



7.2 Use-Case Diagram 2: Faculty and Student Schedule Access

This diagram captures consumer roles accessing published schedules and applying multi-dimensional filters.



7.3 Use Case Specification: UC-GEN-01 (Execute Automated Generation)

This specification describes the primary use case shown in Use-Case Diagram 1.

Use Case ID	UC-GEN-01
Use Case Name	Execute Automated Generation
Primary Actor	Academic Coordinator
Preconditions	Active semester courses, faculty assignments, and rooms are populated in the database with zero unassigned dependencies.
Trigger	Coordinator selects "Generate Master Schedule" from the dashboard.
Postconditions	A complete, conflict-free timetable is staged for coordinator review and manual fine-tuning.

Main Success Scenario

1. System compiles resource arrays (Courses, Faculty, Rooms, Sections) and active constraint parameters.
2. Asynchronous CP-SAT solver job initializes.
3. Solver validates all hard constraints (no double-booking, room capacity, lab blocks).
4. Solver applies soft constraint penalties (minimizing idle time windows).
5. System persists the generated schedule into the database and returns a success response with generation metrics (runtime in seconds, soft penalty score).
6. The master grid displays the generated timetable.

Alternative Flow (Infeasible Configuration)

3a. Constraints conflict mathematically (e.g., required lab hours exceed available physical lab room hours).

3b. System halts, sets status to INFEASIBLE, and displays a diagnostic log highlighting the bottleneck resource.

8. Requirements Traceability Matrix (RTM)

Req ID	Requirement short	Section ref / Design Spec	Module	Test case(s)	Status (N/P/A)	Comments
TT-F-001	Manage Courses & Credits	SAD Sec 3.4 / DB-Course	Resource Management	TC-RES-01	P	Base entity schema
TT-F-002	Manage Faculty & Availability	SAD Sec 3.4 / DB-Faculty	Resource Management	TC-RES-02	P	Base entity schema

TT-F-003	Manage Classrooms & Labs	SAD Sec 3.4 / DB-Room	Resource Management	TC-RES-03	P	Room capacity validation
TT-F-004	Manage Sections & Enrollment	SAD Sec 3.4 / DB-Section	Resource Management	TC-RES-04	P	Batch grouping
TT-F-005	Enforce No Faculty Clashes	SAD Sec 3.5 / Solver-CST	CP-SAT Solver Engine	TC-CST-01	P	Hard constraint
TT-F-006	Enforce No Room Clashes	SAD Sec 3.5 / Solver-CST	CP-SAT Solver Engine	TC-CST-02	P	Hard constraint
TT-F-007	Section Clash & Capacity Checks	SAD Sec 3.5 / Solver-CST	CP-SAT Solver Engine	TC-CST-03	P	Hard constraint
TT-F-008	Consecutive Laboratory Blocks	SAD Sec 3.5 / Solver-CST	CP-SAT Solver Engine	TC-CST-04	P	Multi-slot continuity
TT-F-009	Automated Master Schedule Solving	SAD Sec 4.3 / API-Solve	CP-SAT Solver Engine	TC-GEN-01	P	Core generation API
TT-F-010	Infeasibility Bottleneck Diagnostic	SAD Sec 3.5 / Solver-Diag	CP-SAT Solver Engine	TC-GEN-02	P	Error handling
TT-F-011	Soft Constraint Optimization	SAD Sec 3.5 / Solver-Obj	CP-SAT Solver Engine	TC-GEN-03	P	Idle time minimization
TT-F-012	Multi-View Filterable Timetable Grid	SAD Sec 4.5 / UI-Grid	Web UI / Client	TC-UI-01	P	React / CSS grid view

TT-F-013	Manual Drag-and-Drop Reassignment	SAD Sec 4.3 / API-Reassign	Schedule Editor	TC-MOD-01	P	Includes real-time clash check
TT-F-014	Lock / Pin Session Slots	SAD Sec 3.4 / DB-Schedule	Schedule Editor	TC-MOD-02	P	Domain bounding
TT-F-015	Export Timetable (PDF / Excel)	SAD Sec 4.3 / API-Export	Export Service	TC-EXP-01	P	Document generation
TT-NF-001	Performance ≤ 15 seconds	SAD Sec 3.1 / Arch-Perf	CP-SAT Solver Engine	TC-PERF-01	P	NFR validation
TT-NF-002	System Uptime ≥ 99.5%	SAD Sec 3.1 / Arch-Rel	Infrastructure	TC-REL-01	P	Uptime SLA
TT-NF-003	WCAG 2.1 AA Accessibility	SAD Sec 4.5 / UI-Access	Web UI / Client	TC-UX-01	P	UI contrast & keyboard accessibility
TT-NF-004	Async Solver Architecture	SAD Sec 3.5 / Arch-Async	Backend API	TC-PERF-02	P	Prevents connection timeout
TT-NF-005	Audit Log Retention	SAD Sec 4.4 / Audit-Log	Persistence Layer	TC-AUD-01	P	Change tracking
TT-SR-001	Mandatory TLS 1.3 Encryption	SAD Sec 3.9 / Sec-TLS	Network Layer	TC-SEC-01	P	Secure communications
TT-SR-002	Role-Based Access Control (RBAC)	SAD Sec 3.9 / Sec-Auth	Auth Service	TC-SEC-02	P	Route protection
TT-SR-003	Password	SAD Sec 3.9 /	Auth Service	TC-SEC-03	P	Credential

	Hashing (Argon2 / BCrypt)	Sec-Hash				protection
TT-SR-004	SQLi / XSS Input Sanitization	SAD Sec 3.9 / Sec-Input	API Gateway / Controllers	TC-SEC-04	P	OWASP compliance
TT-SR-005	API Rate Limiting	SAD Sec 3.9 / Sec-Rate	API Gateway / Middleware	TC-SEC-05	N	DoS mitigation